

作业 2

——Max-Cut——

1. 题目

描述

一张图 (V, E) 的割 S 定义为边集 $\{(u, v) \in E : u \in S, v \notin S\}$ 。

请找一个割 S ，使得割的大小不少于总边数的 0.45 倍。

输入

输入数据第一行为数据组数 $T(T \leq 100)$ 。

接下来分别是 T 组数据。

每组数据第一行为正整数 n 和正整数 $m(n \leq 100, m \leq 2000)$ 。

之后 m 行，每行两个数 u 和 v ，为一条边的两个端点。可能有重边，但保证无自环。

输出

包含 T 组数据。

每组数据第一行为正整数 m ，表示 S 的大小。

第二行 m 个数，为 S 中点的编号。可按照任意顺序。

样例输入

```
1
4 4
1 2
2 3
3 4
4 1
```

样例输出

```
2
1 3
```

2. 思路

这个题目如果使用暴力枚举所有割的情况，时间复杂度是 $O(2^n)$ ，对于较大的 n 来说是不可行的。注意到我们输出的结果只需要是一个大于0.45倍总边数的割，因此我们可以使用随机算法来找到一个近似解。

假设所有的割是随机分布的，并且我们讲所有点随机地放到两个集合中，那么产生的割的数量的期望 $\mathbb{E} = 0.5m$ ，其中 m 是边数。期望值已经高于题目要求的0.45，因此，我们只需要多次试验，找到一个大于 $0.45m$ 的割即可。但由于总的割的数目并不确定，我们无法判断找到的这个结果是否达到要求，因此，我们需要数学分析出：进行 N 次试验后，找到一个大于 $0.45m$ 的割的概率。当这个概率足够大的时候，我们认为这件事是一定能办到的。

3. 算法实现

抛开数学不谈，我们假定试验次数为`const int TIMES = 300`，如果精度不够我们只需要调整这个数据取值即可。

3.1. 算法流程

由于这个题目要求进行 T 次输入，每次都几乎完全一样，因此，我们可以把操作放在一个 `while` 循环中：

```
int T;
cin >> T;
while(T--){
    // 每一轮都需要的操作
}
```

对于每一轮操作，我们先读取 n 和 m ，分别代表点的编号以及接下来总的边数；

然后输入每个边上的点对。我们可以使用 `pair<int, int>` 来储存每条边的两个端点，使用 `vector<pair<int, int>>` 来储存所有的边。

接下来，我们需要对每个点进行分组，我们不需要使用两个 `vector` 去记录每个组里面有什么元素，而是可以通过新建一个 `vector<int> flag` 来记录每个点所在的组别，0 代表第一组，1 代表第二组。

得到分组的结果之后，我们需要统计在这种分组结果之下能产生多少个割。我们只需要看看每个边的两个端点是否在同一个组里面，也就是它们的值是否相同，如果不同，那么这个边就会产生一个割，我们就把这个边的数量加 1。这里我新学到了亦或的用法，`[u] ^ flag[v]` 的结果如果是 1，说明它们在不同的组里面，如果是 0，说明它们在同一个组里面。

最后，我们来看看这个分组方法下是不是已有的所有分组方法里面最优的，如果是，我们需要记录产生的割的数目，以及用一个 `vector<int> ans` 来记录这个分组方法下的割的点的编号。

3.2. 代码实现

```
#include <bits/stdc++.h> //在实际写代码的时候，这不是一个好习惯，因为这会给编译器带来一些麻烦。
using namespace std;    //这同样不是个很好的习惯，更好的是写std::cout, std::cin等等。
const int TIMES = 300;
int main()
{
    int T;
    cin >> T;
    srand(time(0));
    while (T--)
    {
        int n, m; // n points, m inputs
        cin >> n >> m;
        vector<pair<int, int>> edge;
        for (int i = 1, v, u; i <= m; i++)
        {
            cin >> v >> u;
            edge.push_back({v, u});
        }
        vector<int> flag(n, 0), ans(n, 0);
        int maxn = 0; // 300 random attempts to find a good solution
        while (TIMES--)
        {
            for (int i = 0; i < n; i++)
                flag[i] = rand() % 2; // Randomly assign each point to group 0 or 1
            int sum = 0;
            for (int i = 0; i < m; i++)
            {
                if (flag[edge[i].first - 1] ^ flag[edge[i].second - 1])
                    sum++;
            }
            if (sum > maxn) // Refresh the best solution found so far
```

```

        {
            maxn = sum;
            for (int i = 0; i < n; i++)
                ans[i] = flag[i];
        }
    }
    int num = 0;
    for (int i = 0; i < n; i++)
        num += ans[i];
    cout << num << endl;
    for (int i = 0; i < n; i++)
        if (ans[i])
            cout << i + 1 << " ";
    cout << endl;
}
return 0;
}

```

3.3. 一点补充

这里的随机数其实是“伪随机”。事实上，要生成一个真随机数是必须要借助物理过程才能实现的，比如说放射性衰变、热噪声等等。我们这里使用的随机数生成器是基于算法的，虽然它们在统计上表现得像随机数，但它们是完全确定的，只要知道了初始状态（种子），你就可以预测出所有的随机数。因此，我们通常称它们为“伪随机数”。“伪随机”也有优劣之分，在这个课程当中，对随机性要求并不会很高，因此，一般的随机数生成函数几乎都可以满足要求。